

A
Project
Report On
MULTITHREADED CHAT APPLICATION

Submitted in partial fulfillment of the requirement for the V semester

**Bachelor of Technology
In
Computer Science and Engineering**

Under the Guidance of
Mr. Aviral Awasthi
Department of CSE



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GRAPHIC ERA HILL UNIVERSITY, BHIMTAL CAMPUS
SATTAL ROAD, P.O. BHOWALI,
DISTRICT- NAINITAL-263132
2025- 2026**

TABLE OF CONTENTS

DECLARATION.....	I
CERTIFICATE.....,.....	II
ACKNOWLEDGEMENT.....	III
PROJECT ABSTRACT.....	IV

CHAPTER 1: INTRODUCTION

- 1.1 Objective
- 1.2 Background
- 1.3 Motivation

CHAPTER 2: SYSTEM FEATURES

CHAPTER 3: SYSTEM ANALYSIS

- 3.1 Problem Statement
- 3.2 Proposed Solution
- 3.3 System Architecture
- 3.4 Flow Diagram

CHAPTER 4: RESOURCES AND TECHNOLOGY USED

CHAPTER 5: CODE AND SCREENSHOTS

CHAPTER 6: FUTURE ENHANCEMENTS AND CONCLUSION

STUDENT'S DECLARATION

We hereby declare the work, which is being presented in the project, entitled **“Multithreaded Chat Application”** in fulfillment of the requirement for the award of the degree **B.Tech** in the session **2025-2026**, is an authentic record of our own work carried out under the supervision of **“Mr. Aviral Awasthi ”, Department of CSE, Graphic Era Hill University, Bhimtal**. The matter embodied in this project has not been submitted by us for the award of any other degree.

Farhan Alam (2361162)

Mayank Joshi (2361326)

Faizan Ali Ansari (2361161)

Shazil Khan (2361499)

CERTIFICATE

The project report entitled “ **Multithreaded Chat Application**” being submitted by **Farhan Alam, Mayank Joshi, Faizan Ali Ansari and Shazil Khan** to Graphic Era Hill University Bhimtal Campus for the award of bonafide work carried out by them. They have worked under my guidance and supervision and fulfilled the requirement for the submission of report.

Mr. Aviral Awasthi
(Project Guide)

Dr. Ankur Bisht
(HOD, CSE Dept.)

ACKNOWLEDGEMENT

We take immense pleasure in thanking Honorable **“Mr. Aviral Awasthi”** (Professor ,CSE, GEHU Bhimtal Campus) to permit me and carry out this project work with her excellent and optimistic supervision. This has all been possible due to her novel inspiration, able guidance and useful suggestions that helped me to develop as a creative researcher and complete the research work, in time.

Words are inadequate in offering my thanks to GOD for providing me everything that we need. We again want to extend thanks to our President **“Prof. (Dr.) Kamal Ghanshala”** for providing us all infrastructure and facilities to work in need without which this work could not be possible.

Many thanks to Professor **“Dr. A.K. Nair”** (Director Gehu Bhimtal), other faculties for their insightful comments, constructive suggestions, valuable advice, and time in reviewing this thesis.

Finally, yet importantly, we would like to express my heartiest thanks to our beloved parents, for their moral support, affection and blessings. We would also like to pay our sincere thanks to all our friends and well-wishers for their help and wishes for the successful completion of this research.

Farhan Alam (2361162)

Mayank Joshi (2361326)

Faizan Ali Ansari (2361161)

Shazil Khan (2361499)

PROJECT ABSTRACT

In today's world, there is a greater need than ever before for an accurate communication system that works in real-time. This is a multithread chat application using Python (Flask) and WebSockets for realtime messaging among users. The system is equipped with several features, including secure user registration, Groupchat's and Privat chat's among the users with private chat rooms, Admin can give access to the same room to multiple users while no other user have access to the room. In addition, it allows users to share files with each other, and video/voice calling through WebRTC. Some additional features are Ollama AI integration which provides smart AI-based replies, bad-words blocking to keep you a clean and peaceful atmosphere in the chats. Real-time magic is delivered by Flask-SocketIO and to keep track of our many clients, we use threads in Python. On the backend, SQLite is used to save the information related to messages and rooms, as well as users' details.

1. INTRODUCTION

Messaging applications have become an integral part of our lives for personal, academic, and professional communications. Our project offers a platform for real-time communication of multiple users via a Flask-based multithreaded chat application with WebSockets.

1.1 OBJECTIVE

- Create a secure, responsive chat system that operates in real time.
- Utilize OS concepts such as multithreading and concurrency in a practical setting.
- Enhance user experience through AI-driven responses and multimedia communication.
- Implement private and group messaging between users with secure protocols and peaceful environment by blocking the profanity or abusive content.

1.2 BACKGROUND

Most of the chat systems work on a single thread which is a traditional and inefficient method as the systems start to lag when multiple users join and the message delivery becomes inconsistent. On the other hand, WebSocket technology keeps the connection open, allowing instant updates and reducing server overhead, thus making it more reliable.

1.3 MOTIVATION

The motivation for this project arises from the increasing demand for reliable, secure, and real-time communication tools. Whatever personal, professional or academic life may be, users expect smooth and instant messaging. We saw a chance to investigate how operating system concepts could be used to solve this problem by creating a fully functional and intelligent chat system.

2. SYSTEM FEATURES

The system provides a range of features:

- **Secure User Authentication:** Users can register via OTP sent to the email using Gmail SMTP. Credentials are stored in a database using hashing algorithms and it prevents any unauthorized access.
- **Public and Private Chat rooms:** Different chat rooms like Global, Tech, and Gaming for group chatting. Separate private chats for targeted communication among users. Also, users can create their own private rooms.
- **Thread-based Client Handling:** Each user is handled in a separate thread by the concept of multithreading so that the users can interact simultaneously without interfering with each other's messages.
- **Instant messages with WebSockets:** Through a persistent WebSocket connection, messages are transferred and received thus ensuring a lag-free experience without the need for page reloads.
- **AI-based responses:** With the help of Ollama AI integration, users can get AI responses by using the /ai keyword.
- **Media sharing:** Documents, images and voice recordings can be shared.
- **Live video/voice calling:** Voice and video calls are made possible through WebRTC to offer users real-time communication.

3. SYSTEM ANALYSIS

3.1 PROBLEM STATEMENT

Conventional chat applications have several drawbacks when practically used:

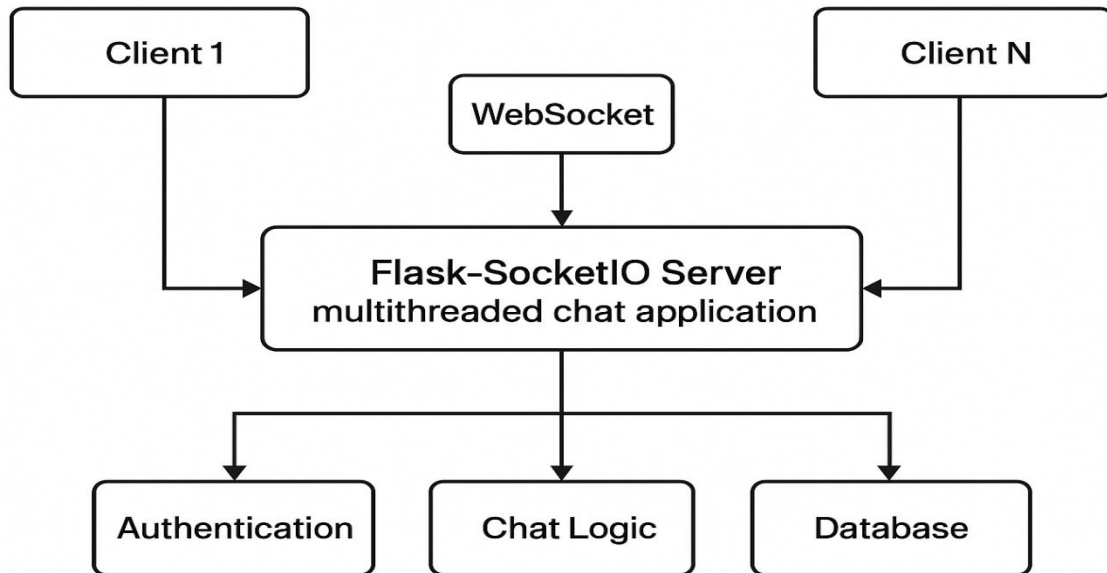
- **Delayed Message Propagation:** Traditional request-response models create unnecessary delays in messaging.
- **Scalability Bottlenecks:** Systems that handle clients one after another may become heavily loaded with the increase of users.
- **Limited User Management:** The absence of role-based access and moderation tools makes chat systems susceptible to spam and misuse.
- **Lack of Multimedia and AI Tools:** Traditional applications do not have modern features like file uploads, automated replies, or voice communication.
- **Security Gaps:** Insecure storage of passwords and weak session control can lead to data breaches or unauthorized access.

3.2 PROPOSED SOLUTION

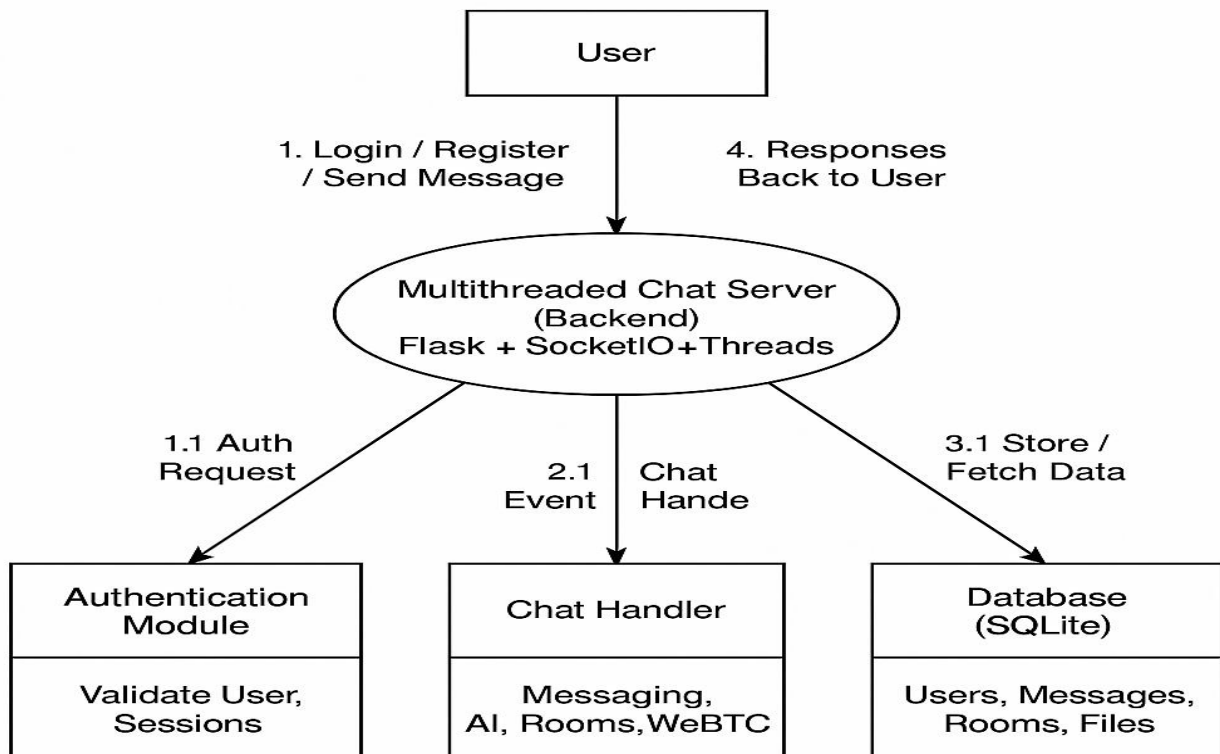
To deal with those problems, this project proposes a solution that is a mixture of multithreading, Flask, and WebSockets technologies:

- **Multithreaded Architecture:** The backend server handles each client in a separate thread, thus avoiding performance lag and enabling concurrency in communication.
- **Persistent Connections with WebSockets:** This method abolishes the need for constant polling by keeping a live connection between the two communicating parties thus achieving real-time message exchange.
- **Secure User Management:** Passwords are hashed by SHA-256 and sessions are stored securely. Access codes can be used to protect each chat room.
- **AI Integration:** The app employs an external Ollama AI to generate helpful, auto-created responses.
- **Voice and Media Features:** The addition of file-sharing and WebRTC-based calling functionalities not only enhances communication but also makes the user experience more engaging with the introduction of the audio feature.
- **Content Filtering:** Provision of an automated profanity filter helps create a safe and peaceful environment for users.

3.3 SYSTEM ARCHITECTURE



3.4 DATA FLOW DIAGRAM



4. RESOURCES AND TECHNOLOGY USED

➤ Programming Languages:

- 1) Python: Server-side logic, threading, socket communication, AI integration.
- 2) HTML5/CSS3: Layout and styling of the web interface.
- 3) JavaScript: Client-side logic for handling events, message updates, media recording, and calling.

➤ Backend Technologies:

- 1) Flask: Web routing, session handling, server-side templating.
- 2) Flask-SocketIO: WebSocket support, real-time bi-directional communication.
- 3) Python `threading`: Concurrent user sessions and message processing.
- 4) SQLite3: A small, embedded SQL database that keeps records of users, chats, and media logs.
- 5) Werkzeug: Implementation of secure password hashing and session protection.
- 6) Requests: Used for interacting with external APIs.

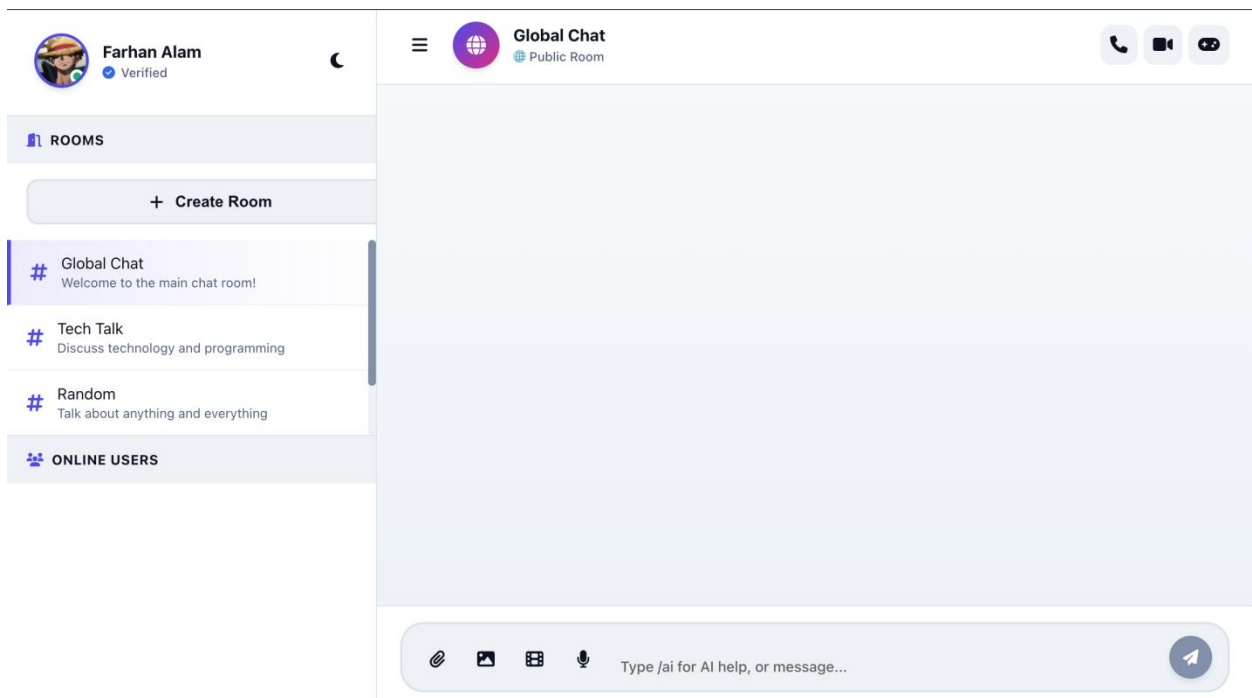
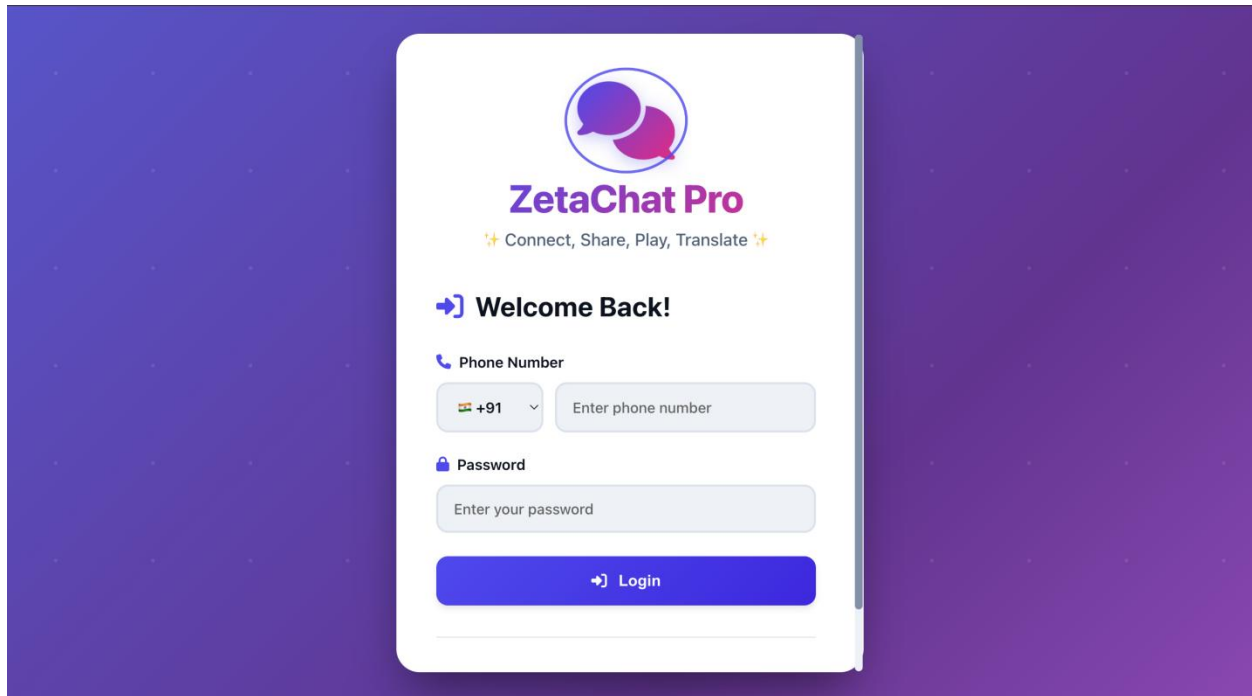
➤ Frontend Technologies:

- 1) Vanilla JavaScript: Minimal scripting for real-time interactions and DOM updates.
- 2) WebRTC API: Holds up audio calls and peer-to-peer communication within the browser.
- 3) Bootstrap (optional): For creating responsive designs and UI components.

➤ Security Mechanisms:

- 1) SHA-256 password hashing.
- 2) User verification through sessions.
- 3) Profanity filtering.

5. WORKING AND SCREENSHOTS



6. FUTURE ENHANCEMENTS AND CONCLUSION

Planning for a feature set that not only broadens the scope of their app but also gears it towards real-world deployment, this is what they have in mind:

- Mobile App Support: Android/iOS native apps created with Flutter or React Native that have push notifications and offline messaging facilities.
- Cloud Deployment: Using AWS or GCP for hosting which will allow dynamic scaling, quick response times, and better availability.
- Advanced Encryption: Complete encryption for all texts just like Signal or WhatsApp ensuring that only the senders and the recipients can read them.
- Two-Factor Authentication: Security layer addition at the time of signing-in by OTPs or authenticator apps.
- Smart Moderation with AI: Using NLP and machine learning models to automatically flag or remove offensive content in the form of texts that are sent by users.
- Video Calls and Screen Sharing: Enhanced WebRTC support for live video, group calls, and screen-sharing functionality.
- Improved Storage: Moving to scalable databases such as PostgreSQL or MongoDB that can handle millions of records in a more efficient way.
- Extended UI Features: Light/dark mode toggle, emoji support, message pinning, user blocking, and downloadable chat histories.

CONCLUSION

The Multithreaded Chat Application is an example of how theoretical concepts from Operating Systems and networking can be used to solve real-world problems. It manages to offer a platform that is scalable and supports real-time communication along with a wide range of features. The application, which has been designed with both instructional and practical purposes in mind, is an excellent example of how multithreading and modern web technologies can be leveraged to enhance digital communication. By further developing this system, it may be able to compete with the platforms that are currently widely used in the communication technology.